ALPE FORMACIÓN · SANTANDER

Edición 2026 · PHP 8.x

CURSO DE PHP · CLASE 18

# PHP moderno y POO avanzada

- Tipos que admiten null y tipos unión
- El operador nullsafe ?→
- Enumeraciones: una lista cerrada de valores
- Enums con valor: guardar y recuperar
- Clases readonly: objetos que no cambian
- __toString, comparar y clonar objetos
- Excepciones propias
- Los errores típicos de hoy



**Ramón Abramo**

+25 años de experiencia docente · Santander, Cantabria

ALPE

FORMACIÓN PROFESIONAL

# 🧭 Contenido de la clase

Si alguien os dice que PHP es un lenguaje viejo y desordenado, es que no lo ha tocado en diez años. Las versiones 8.x han traído herramientas que hacen el código **más seguro y más fácil de leer**: tipos que admiten `null`, enumeraciones, objetos que no se pueden modificar...

Hoy vemos las que más vais a usar en el día a día, y en Laravel. Ninguna es imprescindible para que un programa funcione, pero todas sirven para que **falle menos**. Y eso, cuando el programa es vuestro y suena el teléfono un domingo, se agradece mucho. 😅

**1**

📘 **Tipos que admiten null y tipos unión**



**2**

🔍 **El operador nullsafe ?→**



**3**

👌 **Enumeraciones: una lista cerrada de valores**



**4**

👍 **Enums con valor: guardar y recuperar**



**5**

⚠️ **Clases readonly: objetos que no cambian**



**6**

😮 **__toString, comparar y clonar objetos**



**7**

🐞 **Excepciones propias**



**8**

🐞 **Los errores típicos de hoy**



**9**

🚀 **Ejercicios**



**10**

👋 **Lo que tenéis que llevaros de hoy**





CLASE 18 · PUNTO 1

## 📘 Tipos que admiten null y tipos unión

Ya lo habéis visto de pasada: un `?` delante de un tipo significa «este tipo **o** `null`». Se usa para datos que pueden faltar, como la nota de un alumno que todavía no se ha examinado [1].

01-tipos-nullable-union.php — php-paso-a-paso/clase18 — Visual Studio Code

01-tipos-nullable-union.php

```php
<?php
class Alumno
{
    public function __construct(
        public string $nombre,
        public ?float $nota = null,
    ) {
    }
}

function saludar(?string $apodo): string
{
    if ($apodo === null) {
        return "Hola, desconocido";
    }
    return "Hola, {$apodo}";
}
```

Y si un dato puede ser de **varios** tipos, se escriben separados por `|`. Es un **tipo unión**:

01-tipos-nullable-union.php — php-paso-a-paso/clase18 — Visual Studio Code

01-tipos-nullable-union.php

```php
<?php
function doble(int|float $numero): int|float
{
    return $numero * 2;
}
```

| Escribís | Significa | Ejemplo de valores |
|---|---|---|
| `string` | Siempre un texto | `"Hola"` |
| `?string` | Un texto o `null` | `"Hola"`, `null` |
| `int\|float` | Un entero o un decimal | `4`, `2.5` |
| `mixed` | Cualquier cosa (úsalo poco) | lo que sea |

📄 Ejemplo: `01-tipos-nullable-union.php`



🎯 Cuanto más preciso sea el tipo, antes os avisará PHP de un error. Usad `?` solo cuando `null` tenga un significado real («todavía no hay nota»), no por comodidad.



CLASE 18 · PUNTO 2

## 🔍 El operador nullsafe ?→

Imaginad un usuario que puede tener dirección... o no. Si no la tiene y escribís `$usuario→getDireccion()→ciudad`, PHP se queja: *Attempt to read property "ciudad" on null*. Hasta ahora había que comprobarlo con un `if`:

02-nullsafe.php — php-paso-a-paso/clase18 — Visual Studio Code

02-nullsafe.php

```php
<?php
$ciudad = "desconocida";
if ($usuario→getDireccion() !== null) {
    $ciudad = $usuario→getDireccion()→ciudad;
}
```

Con el **operador nullsafe** `?→`, si lo que hay a la izquierda es `null`, el resultado es `null` y no hay error. Y combinado con el `??` de la clase 7, queda en una línea [2]:

02-nullsafe.php — php-paso-a-paso/clase18 — Visual Studio Code

02-nullsafe.php

```php
<?php
$ciudad = $usuario→getDireccion()?→ciudad ?? "desconocida";
```

Leedlo así: «la ciudad de la dirección del usuario, **si tiene**; y si no, "desconocida"».

📄 Ejemplo: `02-nullsafe.php`



💡 No confundáis los dos interrogantes: `??` da un valor por defecto si algo es `null`; `?->` evita el error al **entrar** en algo que puede ser `null`. Suelen ir juntos.



CLASE 18 · PUNTO 3

## 👌 Enumeraciones: una lista cerrada de valores

¿Cuántas veces habéis guardado estados como textos sueltos, tipo `"pendiente"` o `"enviado"`? Basta una errata (`"pendinte"`) para que el programa falle sin avisar. Las **enumeraciones** (*enums*) lo arreglan: definen un tipo con una **lista cerrada** de valores posibles [3].

03-enum-puro.php — php-paso-a-paso/clase18 — Visual Studio Code

03-enum-puro.php

```php
<?php
enum Talla
{
    case S;
    case M;
    case L;
    case XL;
}

function precioCamiseta(Talla $talla): float
{
    return match ($talla) {
        Talla::S, Talla::M => 15.0,
        Talla::L => 17.0,
        Talla::XL => 19.0,
    };
}

echo precioCamiseta(Talla::L);   // 17
precioCamiseta("L");             // ❌ TypeError: un texto no es una Talla
```

| Uso | Cómo |
|---|---|
| Un valor del enum | `Talla::L` |
| Su nombre | `Talla::L->name` → `"L"` |
| Todos los valores | `Talla::cases()` |
| Comparar | `$talla === Talla::L` |

Ya no hay erratas posibles: si escribís `Talla::LL`, PHP os avisa al momento. Y `match` es el compañero perfecto, porque si un día añadís la talla `XXL` y se os olvida ponerla en el `match`, os lo dirá con un `UnhandledMatchError`.

📄 Ejemplo: `03-enum-puro.php`



💡 Escribo los casos en MAYÚSCULAS, como las constantes, porque en el fondo lo son: valores fijos con nombre.



CLASE 18 · PUNTO 4

## 👍 Enums con valor: guardar y recuperar

Un enum como `Talla` está muy bien dentro del programa, pero ¿qué guardamos en la base de datos? Para eso existen los **enums respaldados** (*backed enums*): cada caso lleva un valor, `string` o `int`, que es lo que se guarda [4].

04-enum-respaldado.php — php-paso-a-paso/clase18 — Visual Studio Code

04-enum-respaldado.php

```php
<?php
enum EstadoPedido: string
{
    case PENDIENTE = "pendiente";
    case ENVIADO = "enviado";
    case ENTREGADO = "entregado";
    case CANCELADO = "cancelado";

    public function etiqueta(): string
    {
        return match ($this) {
            EstadoPedido::PENDIENTE => "⏳ Pendiente de envío",
            EstadoPedido::ENVIADO => "🚚 En camino",
            EstadoPedido::ENTREGADO => "✅ Entregado",
            EstadoPedido::CANCELADO => "❌ Cancelado",
        };
    }
}

$estado = EstadoPedido::ENVIADO;
echo $estado->value;        // "enviado" → esto se guarda en la base de datos
echo $estado->etiqueta();   // 🚚 En camino → esto se muestra al usuario
```

Y para el camino de vuelta, del texto guardado al enum:

| Método | Si el texto es válido | Si no lo es |
|---|---|---|
| `EstadoPedido::from("enviado")` | `EstadoPedido::ENVIADO` | Lanza un `ValueError` |
| `EstadoPedido::tryFrom("perdido")` | El caso correspondiente | Devuelve `null` |



🎯 **Mi regla:** `from()` para lo que viene de **vuestra** base de datos (debería ser siempre válido); `tryFrom()` para lo que viene de un **formulario** o de la URL, porque nunca os fiáis del usuario.

Fijaos también en que **un enum puede tener métodos**, como `etiqueta()`. Así, todo lo que tiene que ver con los estados está en un solo sitio.

📄 Ejemplo: `04-enum-respaldado.php`



CLASE 18 · PUNTO 5

# ⚠️ Clases readonly: objetos que no cambian

En la clase 16 visteis propiedades `readonly`. Desde PHP 8.2 podéis marcar **la clase entera**: todas sus propiedades pasan a ser de solo lectura y el objeto se vuelve **inmutable** [5].

¿Para qué queremos un objeto que no se puede cambiar? Para valores que deben ser fiables, como una cantidad de dinero. Si un precio pudiera modificarse sin querer en cualquier parte del programa, ¿cómo os fiaríais del total de una factura?

05-readonly-class.php — php-paso-a-paso/clase18 — Visual Studio Code

05-readonly-class.php

```php
<?php
readonly class Dinero
{
    public function __construct(
        public int $centimos,
        public string $moneda = "EUR",
    ) {
    }

    public static function euros(float $cantidad): Dinero
    {
        return new Dinero((int) round($cantidad * 100));
    }

    public function sumar(Dinero $otro): Dinero
    {
        return new Dinero($this->centimos + $otro->centimos, $this->moneda);
    }
}

$precio = Dinero::euros(19.99);
$total = $precio->sumar(Dinero::euros(4.95));   // un objeto NUEVO
// $precio sigue valiendo 19,99 €
```

Como no se puede modificar, `sumar()` no cambia el objeto: **devuelve uno nuevo** con el resultado, igual que `3 + 2` no cambia el 3.

📄 Ejemplo: `05-readonly-class.php`



💡 ¿Por qué guardar **céntimos** como entero en lugar de euros con decimales? Porque los ordenadores no representan exactamente algunos decimales (probad `var_dump(0.1 + 0.2);` y veréis `0.30000000000000004`). Con dinero, los céntimos enteros evitan sorpresas.



CLASE 18 · PUNTO 6

# 😮 __toString, comparar y clonar objetos

## __toString: qué se muestra al hacer echo

Si intentáis hacer `echo $producto;` con un objeto normal, PHP se queja. Con el método mágico `__toString()` decidís qué texto representa al objeto [6]:

06-tostring-comparar.php — php-paso-a-paso/clase18 — Visual Studio Code

06-tostring-comparar.php

```php
<?php
class Producto
{
    public function __construct(public string $nombre, public float $precio)
    {
    }

    public function __toString(): string
    {
        return "{$this->nombre} (" . number_format($this->precio, 2, ",", ".") . " €)";
    }
}

echo $taza;   // Taza (7,95 €)
```

## == frente a ===, también con objetos

| Comparación | Con objetos significa… |
|---|---|
| `$a == $b` | ¿Son de la misma clase y tienen **los mismos datos**? |
| `$a === $b` | ¿Son **el mismo objeto**? |

## ¡Ojo! Asignar un objeto no lo copia

Esta es de las que pillan a todo el mundo:

06-tostring-comparar.php — php-paso-a-paso/clase18 — Visual Studio Code

06-tostring-comparar.php

```php
<?php
$taza1 = new Producto("Taza", 7.95);
$mismaTaza = $taza1;          // NO es una copia: es otro nombre para el MISMO objeto
$copia = clone $taza1;        // esto sí es una copia independiente

$mismaTaza->precio = 9.95;
echo $taza1;                  // Taza (9,95 €) → ¡también ha cambiado!
```

Con los arrays y los números, `$b = $a` hace una copia. Con los objetos, no: las dos variables apuntan **al mismo objeto**. Si necesitáis una copia de verdad, `clone` [7].

📄 Ejemplo: `06-tostring-comparar.php`




Hasta ahora lanzábamos `Exception`, `RuntimeException` o `InvalidArgumentException`. Funciona, pero el `catch` no sabe **qué** ha pasado exactamente. Ahora que sabéis heredar, podéis crear **vuestras propias excepciones**, con un nombre que lo dice todo y con los datos que hagan falta [8]:

07-excepciones-propias.php — php-paso-a-paso/clase18 — Visual Studio Code

07-excepciones-propias.php

```php
<?php
class SaldoInsuficienteException extends RuntimeException
{
    public function __construct(private float $saldo, private float $pedido)
    {
        parent::__construct("Saldo insuficiente: tienes {$saldo} € y quieres sacar {$pedido}
€.");
    }

    public function getFalta(): float
    {
        return $this->pedido - $this->saldo;
    }
}

try {
    $cuenta->retirar(500);
} catch (SaldoInsuficienteException $e) {
    echo $e->getMessage() . " Te faltan {$e->getFalta()} €.";
}
```

| Ventaja | Por qué |
|---|---|
| El nombre explica el problema | `catch (SaldoInsuficienteException $e)` se lee solo |
| Trae sus propios datos | `getFalta()` permite ofrecer una solución al usuario |
| Se puede capturar con precisión | Cada tipo de error, en su propio `catch` |

¿Recordáis el aviso de la clase 14? Poníamos `catch (PDOException $e)` **antes** que `catch (RuntimeException $e)`. Ahora sabéis por qué: `PDOException` **hereda** de `RuntimeException`, igual que vuestra `SaldoInsuficienteException`. Un `catch` de la clase padre atrapa también a todas sus hijas, así que los más concretos van primero.

📄 Ejemplo: `07-excepciones-propias.php`



🎯 Por convenio, el nombre de una excepción termina en `Exception`. Y elegid bien de quién hereda: `InvalidArgumentException` para datos incorrectos que llegan a una función, `RuntimeException` para problemas que surgen al ejecutar (falta saldo, falta stock...).



CLASE 18 · PUNTO 8

## 🐞 Los errores típicos de hoy

| Mensaje | Qué ha pasado | Solución |
|---|---|---|
| `Attempt to read property "ciudad" on null` | Entráis en algo que es `null` | `?->` y un valor por defecto con `??` |
| `Argument #1 must be of type Talla, string given` | Pasáis un texto donde se espera un enum | `Talla::L`, o convertid con `from()` / `tryFrom()` |
| `"perdido" is not a valid backing value for enum EstadoPedido` | `from()` con un valor que no existe | Usad `tryFrom()` y comprobad el `null` |
| `Cannot modify readonly property Dinero::$centimos` | Intentáis cambiar un objeto inmutable | Cread un objeto nuevo con el resultado |
| `Object of class Producto could not be converted to string` | `echo` de un objeto sin `__toString()` | Añadid `__toString()` o mostrad una propiedad |
| «He cambiado una variable y ha cambiado otra» | `$b = $a` con objetos no copia | `clone` si queréis una copia independiente |



CLASE 18 · PUNTO 9

# 🚀 Ejercicios

- ☐ **01 · Días de la semana.** Un enum `DiaSemana` respaldado por enteros (1 = lunes… 7 = domingo) con los métodos `nombre()`, `esFinDeSemana()` y `hoy()` (pista: `date("N")`). → `ej01-dias-semana.php`
- ☐ **02 · Estados de un pedido.** Un `Pedido` que empieza `PENDIENTE` y solo puede cambiar a los estados permitidos (de pendiente a enviado o cancelado; de enviado a entregado). Si no, lanza una `TransicionNoPermitidaException`. Guardad el historial de estados. → `ej02-pedido-estados.php`
- ☐ **03 · Dinero.** Ampliad la clase `readonly` `Dinero` con `multiplicar()`, `porcentaje()` y `__toString()`, y calculad una compra con IVA sin modificar ningún objeto. → `ej03-dinero.php`
- ☐ **04 · Clientes incompletos.** Una tabla de clientes en la que algunos no tienen dirección, código postal o teléfono. Ni un solo `if`: solo `?->` y `??`. → `ej04-direcciones.php`
- ☐ **05 · Almacén.** Una clase `Almacen` que lance una `StockInsuficienteException` (con `getDisponibles()`) o una `ProductoNoEncontradoException`, y un programa que las capture y ofrezca alternativas. → `ej05-stock.php`



🏆 **Reto:** añadid una columna `estado` a la tabla de alumnos de la clase 14 (`ALTER TABLE alumnos ADD estado VARCHAR(20) NOT NULL DEFAULT 'activo';`), cread un enum `EstadoAlumno` (activo, baja, graduado) y mostrad cada alumno con la etiqueta de su estado usando `EstadoAlumno::from($fila["estado"])`.



CLASE 18 · PUNTO 10

# 👋 Lo que tenéis que llevaros de hoy

| Idea | En una frase |
|---|---|
| `?tipo` | El tipo o `null`, para datos que pueden faltar |
| `int\|float` | Tipo unión: uno u otro |
| `?->` | Entra en algo solo si no es `null`; con `??`, un valor por defecto |
| `enum` | Una lista cerrada de valores: adiós a las erratas en los textos |
| Enum respaldado | `->value` para guardar; `from()` y `tryFrom()` para recuperar |
| `readonly class` | Objetos inmutables: operar devuelve un objeto nuevo |
| `__toString()` | Lo que se muestra al hacer `echo` del objeto |
| `==` y `===` | Mismos datos / mismo objeto |
| `clone` | Asignar un objeto no lo copia; `clone`, sí |
| Excepciones propias | Heredan de las de PHP, tienen nombre propio y traen sus datos |

# 📚 Fuentes y documentación oficial

1. Manual de PHP: declaraciones de tipo (nullable y union)
2. Manual de PHP: operador nullsafe
3. Manual de PHP: enumeraciones
4. Manual de PHP: enumeraciones respaldadas
5. PHP 8.2: clases readonly
6. Manual de PHP: métodos mágicos (__toString)
7. Manual de PHP: clonación de objetos
8. Manual de PHP: extender excepciones